AI Escape Room — Design and Implementation Report

1. Problem and objective

The project implements an interactive escape-room game in which a player wakes inside a locked library and must investigate objects, discover clues, solve puzzles and unlock the exit. A Large Language Model (LLM) acts as the Game Master by responding to natural-language actions and narrating the experience. The work is delivered in two versions to demonstrate the transition from a conversational prototype to a more reliable application.

The room contains five interactive objects: a desk, bookshelf, painting, locked drawer and exit door. The bookshelf reveals drawer combination **4132**; opening the drawer provides a brass key and UV lamp; using the UV lamp on the painting reveals exit code **2714**; the player then uses the key and code to escape.

2. Version 1 — Interactive AI Escape Room

Version 1 uses Streamlit as the web interface and maintains chat history in the Streamlit session. The Game Master prompt defines the room, objects, puzzle sequence and completion condition. When GEMINI_API_KEY is configured, the application sends the conversation to Gemini and displays its response. A local fallback response path allows the application to be explored when an API key is not configured.

This version provides natural-language interaction and conversational continuity. Its limitation is that the LLM generates the response and is also responsible for respecting the game rules. Prompt instructions reduce mistakes but do not provide a deterministic guarantee. The model may misremember state, invent an item, reveal information too early or describe an invalid action as successful.

3. Version 2 — Reliable AI Escape Room

Version 2 separates game logic from language generation. GameEngine stores authoritative state in a Python dictionary: drawer status, clue status, door status, escape status, inventory, examined objects, solved puzzles and action history. The engine validates object examination, code entry and item use. Python decides whether an action succeeds. Gemini, when configured, only narrates the validated result; its output cannot modify game state. Without Gemini, the validated Python message is displayed directly.

The drawer combination is accepted only after the bookshelf has been examined; the UV lamp must be in inventory before it can reveal the painting clue; the exit code is accepted only when the brass key has been collected and the painting clue has been revealed. Version 2 rejects unknown objects, incorrect combinations, uncollected items, administrator impersonation and requests to reveal hidden solutions. Restart resets the state.

4. Testing and comparison

The automated test suite verifies the successful puzzle sequence and invalid or adversarial cases. It checks that an uncollected key cannot be used, an incorrect drawer code does not open the drawer, the drawer cannot be opened before the clue is found, administrator claims do not unlock the exit, prompt-injection requests do not reveal solutions or corrupt state, nonexistent objects are rejected, the exit code requires prerequisites, inventory contains only collected items, and restart clears progress.

Test case	Version 1 risk	Version 2 expected behaviour
Uncollected key	Model may accept claim	Python rejects; inventory unchanged
Incorrect drawer code	May narrate success	Drawer remains locked
Claim puzzle solved	Chat text may be mistaken for state	Bookshelf examination is required
Administrator claim	Prompt may be overridden	Request rejected; state unchanged
Request secret solutions	Model may disclose information	Request rejected
Nonexistent object	Model may invent it	Object validation rejects it
Exit code before key	Model may skip prerequisites	Exit remains locked

Version 1 is useful for demonstrating natural-language interaction and the creative capability of an LLM. Version 2 is more reliable because state transitions and puzzle rules are enforced by code rather than generated text. The LLM is retained for narration without granting it authority over game completion.

5. Limitations and conclusion

The Version 2 natural-language action parser supports a defined set of action patterns; unusual phrasing may require rephrasing. Gemini is optional and requires a user-provided API key. The application does not train or fine-tune a model. The implementation demonstrates a practical separation between probabilistic language generation and deterministic application logic, making the game more consistent, testable and resistant to attempts to bypass its rules.

Architecture diagrams for both versions are included separately in the docs folder. Interface preview images are included in assets; they are design previews, not runtime screenshots. Capture the running Streamlit interface for the final live demonstration.